LAPORAN PRAKTIKUM

Blynk OTA

Mata Kuliah ROBOTIKA

Nama Dino Rosalino Yuswanto

NPM 23081010152

Informatika Fakultas Ilmu Komputer

1. Ringkasan dan Target Pengujian

1.1 Ringkasan tugas

Praktikum dilakukan dengan mengintegrasikan arduino dengan Blynk.

1.2 Lingkungan pengujian

Komponen	Versi / Spesifikasi
Bahasa / framework	Arduino
Perangkat / simulator	ESP 32 & Blynk

2. Kode Program

2.1 Potongan kode utama

Kode Edgent_ESP32.ino

```
/*************************
  Blynk is a platform with iOS and Android apps to control
  ESP32, Arduino, Raspberry Pi and the likes over the Internet.
  You can easily build mobile and web interfaces for any
  projects by simply dragging and dropping widgets.

    Downloads, docs, tutorials: https://www.blynk.io
    Sketch generator:           https://examples.blynk.cc
    Blynk community:            https://community.blynk.cc
    Follow us:                  https://www.fb.com/blynkapp
                                https://twitter.com/blynk_app

  Blynk library is licensed under MIT license
 *************************
  Blynk.Edgent implements:
  - Blynk.Inject - Dynamic WiFi credentials provisioning
  - Blynk.Air    - Over The Air firmware updates
  - Device state indication using a physical LED
  - Credentials reset using a physical Button
 *************************/

/* Fill in information from your Blynk Template here */
/* Read more: https://bit.ly/BlynkInject */
#define BLYNK_TEMPLATE_ID "TMPL6Zh3pGdg3"
#define BLYNK_TEMPLATE_NAME "Tes"

#define BLYNK_FIRMWARE_VERSION        "0.1.0"

#define BLYNK_PRINT Serial
//#define BLYNK_DEBUG

#define APP_DEBUG

// Uncomment your board, or configure a custom board in Settings.h
//#define USE_ESP32_DEV_MODULE
//#define USE_ESP32C3_DEV_MODULE
//#define USE_ESP32S2_DEV_KIT
//#define USE_WROVER_BOARD
//#define USE_TTGO_T7
//#define USE_TTGO_T_OI
```

```
#include "BlynkEdgent.h"

void setup()
{
  Serial.begin(115200);
  delay(100);

  BlynkEdgent.begin();
}

void loop() {
  BlynkEdgent.run();
}
```

Kode OTA_uafi.ino

```
/*************************************************************
 * iMCLab + ESP32 + Blynk.Edgent + Blynk Air
 * Monitoring RPM, Kontrol PWM, dan Motor ON/OFF
 *
 * V0 = RPM
 * V1 = Nilai PWM
 * V2 = Slider PWM (0-255)
 * V3 = Switch Motor ON/OFF
 *************************************************************/

// Identitas Blynk Template
#define BLYNK_TEMPLATE_ID   "TMPL6Zh3pGdg3"
#define BLYNK_TEMPLATE_NAME "Tes"

// Naikkan setiap membuat firmware baru.
// Gunakan versi yang lebih tinggi dari firmware terakhir.
#define BLYNK_FIRMWARE_VERSION "0.1.2"

#define BLYNK_PRINT Serial
#define APP_DEBUG

#include <esp_arduino_version.h>
#include "BlynkEdgent.h"

// ============================================================
// PIN MOTOR DAN LED
// ============================================================

const int ledPin = 2;

const int motor1Pin1 = 27;
```

```
const int motor1Pin2 = 26;
const int enable1Pin = 12;

// ============================================================
// SENSOR RPM
// ============================================================

const byte pin_rpm = 13;

// Jumlah pulsa sensor untuk setiap satu putaran
const int HOLES = 2;

volatile unsigned long rev = 0;
volatile unsigned long lastPulseTime = 0;

const unsigned long DEBOUNCE_US = 500;

float rpm = 0.0;
unsigned long old_time = 0;

// ============================================================
// KONFIGURASI PWM
// ============================================================

const int freq = 30000;
const int pwmChannel = 0;
const int resolution = 8;

int dutyCycle = 200;

const int DUTY_MIN = 0;
const int DUTY_MAX = 255;
const int DUTY_STEP = 10;

bool isMotorOn = false;
bool motorStateChanged = true;

// ============================================================
// SERIAL DAN TIMER
// ============================================================

String inputString = "";

BlynkTimer timer;

const unsigned long SEND_INTERVAL_MS = 1000;

// ============================================================
```

```
// PWM - KOMPATIBEL DENGAN ESP32 CORE 2.x DAN 3.x
// ============================================================

void setupMotorPWM()
{
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcAttach(enable1Pin, freq, resolution);
#else
  ledcSetup(pwmChannel, freq, resolution);
  ledcAttachPin(enable1Pin, pwmChannel);
#endif

  writeMotorPWM(0);
}

void writeMotorPWM(int duty)
{
  duty = constrain(duty, DUTY_MIN, DUTY_MAX);

#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcWrite(enable1Pin, duty);
#else
  ledcWrite(pwmChannel, duty);
#endif
}

// Nilai PWM aktual yang diberikan ke driver motor
int getActualPWM()
{
  if (isMotorOn && dutyCycle > 0)
    return dutyCycle;

  return 0;
}

// ============================================================
// INTERRUPT SENSOR RPM
// ============================================================

void IRAM_ATTR isr()
{
  unsigned long now = micros();

  if (now - lastPulseTime > DEBOUNCE_US)
  {
    rev++;
    lastPulseTime = now;
  }
```

```
}

// ============================================================
// HITUNG RPM
// ============================================================

void hitungRPM()
{
  unsigned long cur_time = millis();
  unsigned long elapsed = cur_time - old_time;

  if (elapsed >= 1000)
  {
    // Mengambil jumlah pulsa dengan menghentikan interrupt
    // sesaat untuk menghindari konflik saat membaca/reset.
    detachInterrupt(digitalPinToInterrupt(pin_rpm));

    unsigned long pulseCount = rev;
    rev = 0;

    attachInterrupt(
      digitalPinToInterrupt(pin_rpm),
      isr,
      FALLING
    );

    // RPM = pulsa / pulsa per putaran / waktu (menit)
    rpm = ((float)pulseCount * 60000.0f) /
          ((float)HOLES * (float)elapsed);

    old_time = cur_time;
  }
}

// ============================================================
// TERAPKAN PWM DAN ARAH MOTOR
// ============================================================

void applyMotor()
{
  int actualPWM = getActualPWM();

  if (actualPWM > 0)
  {
    // Arah motor seperti konfigurasi awal
    digitalWrite(motor1Pin1, HIGH);
    digitalWrite(motor1Pin2, LOW);
  }
```

```
  else
  {
    // Motor OFF
    digitalWrite(motor1Pin1, LOW);
    digitalWrite(motor1Pin2, LOW);
  }

  writeMotorPWM(actualPWM);

  // LED hanya menyala ketika PWM aktual > 0
  digitalWrite(ledPin, actualPWM > 0 ? HIGH : LOW);

  // Sinkronisasi kontrol jika Blynk terhubung
  if (Blynk.connected())
  {
    Blynk.virtualWrite(V2, dutyCycle);
    Blynk.virtualWrite(V3, isMotorOn ? 1 : 0);
  }
}

// ============================================================
// PERBARUI HARDWARE JIKA ADA PERUBAHAN
// ============================================================

void updateDevice()
{
  if (motorStateChanged)
  {
    applyMotor();
    motorStateChanged = false;
  }
}

// ============================================================
// KONEKSI BLYNK
// ============================================================

BLYNK_CONNECTED()
{
  Serial.println("Blynk terhubung.");

  // Keamanan: motor tidak otomatis hidup saat reconnect
  isMotorOn = false;
  motorStateChanged = true;

  // Atur switch dashboard ke OFF
  Blynk.virtualWrite(V3, 0);
```

```
  // Ambil nilai slider PWM tersimpan dari Blynk
  Blynk.syncVirtual(V2);
}

// ============================================================
// V2: SLIDER PWM (0-255)
// ============================================================

BLYNK_WRITE(V2)
{
  dutyCycle = constrain(
    param.asInt(),
    DUTY_MIN,
    DUTY_MAX
  );

  // Jika PWM 0, matikan motor
  if (dutyCycle == 0)
  {
    isMotorOn = false;
  }

  motorStateChanged = true;

  Serial.print("PWM dari Blynk: ");
  Serial.println(dutyCycle);
}

// ============================================================
// V3: SWITCH MOTOR ON/OFF
// ============================================================

BLYNK_WRITE(V3)
{
  bool requestedState = (param.asInt() != 0);

  if (requestedState)
  {
    if (dutyCycle == 0)
    {
      dutyCycle = 200;
    }

    isMotorOn = true;
  }
  else
  {
    // Motor mati, nilai slider tetap tersimpan
```

```
    isMotorOn = false;
  }

  motorStateChanged = true;

  Serial.print("Motor ");
  Serial.println(isMotorOn ? "ON" : "OFF");
}

// ============================================================
// KIRIM DATA KE BLYNK
// ============================================================

void kirimKeBlynk()
{
  if (!Blynk.connected())
    return;

  // V0 = RPM aktual dari sensor
  Blynk.virtualWrite(V0, rpm);

  // V1 = PWM aktual yang diterapkan ke motor.
  // Nilainya 0 ketika motor OFF.
  Blynk.virtualWrite(V1, getActualPWM());
}

// ============================================================
// STATUS SERIAL
// ============================================================

void tampilkanStatus(const char* aksi)
{
  Serial.print(aksi);
  Serial.print(" | RPM=");
  Serial.print(rpm, 1);

  Serial.print(" | PWM Set=");
  Serial.print(dutyCycle);

  Serial.print(" | PWM Aktual=");
  Serial.print(getActualPWM());

  Serial.print(" | Motor=");
  Serial.println(isMotorOn ? "ON" : "OFF");
}

// ============================================================
// PEMROSESAN PERINTAH SERIAL
```

```
// ============================================================

void prosesSerial()
{
  while (Serial.available() > 0)
  {
    char c = Serial.read();

    if (c == '\n' || c == '\r')
    {
      inputString.trim();

      if (inputString.startsWith("PWM:"))
      {
        int val = inputString.substring(4).toInt();

        dutyCycle = constrain(
          val,
          DUTY_MIN,
          DUTY_MAX
        );

        isMotorOn = (dutyCycle > 0);
        motorStateChanged = true;

        tampilkanStatus("Set PWM");
      }
      else if (inputString.length() > 0)
      {
        char cmd = inputString.charAt(0);

        if (cmd == '1')
        {
          if (dutyCycle == 0)
            dutyCycle = 200;

          isMotorOn = true;
          motorStateChanged = true;

          tampilkanStatus("Motor ON");
        }
        else if (cmd == '0')
        {
          isMotorOn = false;
          motorStateChanged = true;

          tampilkanStatus("Motor OFF");
        }
```

```
        else if (cmd == '+')
        {
          dutyCycle = constrain(
            dutyCycle + DUTY_STEP,
            DUTY_MIN,
            DUTY_MAX
          );

          if (dutyCycle > 0)
            isMotorOn = true;

          motorStateChanged = true;

          tampilkanStatus("PWM NAIK");
        }
        else if (cmd == '-')
        {
          dutyCycle = constrain(
            dutyCycle - DUTY_STEP,
            DUTY_MIN,
            DUTY_MAX
          );

          if (dutyCycle == 0)
            isMotorOn = false;

          motorStateChanged = true;

          tampilkanStatus("PWM TURUN");
        }
        else if (cmd == 'R' || cmd == 'r')
        {
          tampilkanStatus("Status");
        }
      }

      inputString = "";
    }
    else if (inputString.length() < 40)
    {
      inputString += c;
    }
  }
}

// ============================================================
// SETUP
// ============================================================
```

```
void setup()
{
  Serial.begin(115200);
  delay(100);

  // Inisialisasi pin
  pinMode(ledPin, OUTPUT);

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(enable1Pin, OUTPUT);

  pinMode(pin_rpm, INPUT_PULLUP);

  // Pastikan motor mati ketika boot
  digitalWrite(motor1Pin1, LOW);
  digitalWrite(motor1Pin2, LOW);
  digitalWrite(ledPin, LOW);

  setupMotorPWM();

  attachInterrupt(
    digitalPinToInterrupt(pin_rpm),
    isr,
    FALLING
  );

  old_time = millis();

  // Timer pengiriman RPM dan PWM
  timer.setInterval(
    SEND_INTERVAL_MS,
    kirimKeBlynk
  );

  Serial.println("==============================");
  Serial.println("iMCLab + Blynk.Edgent");
  Serial.println("RPM + PWM + Motor Control");
  Serial.print("Firmware versi: ");
  Serial.println(BLYNK_FIRMWARE_VERSION);
  Serial.println("==============================");
  Serial.println("1       = Motor ON");
  Serial.println("0       = Motor OFF");
  Serial.println("+       = PWM naik");
  Serial.println("-       = PWM turun");
  Serial.println("PWM:200 = Set PWM dan hidupkan motor");
  Serial.println("R       = Lihat RPM dan PWM");
```

```
  Serial.println("==============================");

  // Koneksi Edgent dan pengelolaan Blynk Air
  BlynkEdgent.begin();
}

// ============================================================
// LOOP
// ============================================================

void loop()
{
  // Koneksi Blynk dan mekanisme Blynk Air
  BlynkEdgent.run();

  // Pengukuran RPM dan pengiriman data
  hitungRPM();
  timer.run();

  // Perintah kontrol melalui Serial Monitor
  prosesSerial();

  // Terapkan perubahan motor
  updateDevice();
}
```

3. Penjelasan Kode

3.1 Folder Edgent_ESP32

Folder Edgent_ESP32 berisi program dasar yang terlebih dahulu dimasukkan ke ESP32. Program ini berfungsi sebagai firmware awal yang menyiapkan perangkat agar dapat terhubung dengan platform Blynk dan menerima pembaruan firmware melalui fitur Over-The-Air (OTA) pada Blynk.Air. Dengan demikian, folder ini bukan program utama pengendalian motor yang menjadi tujuan akhir praktikum, melainkan program dasar yang menyediakan mekanisme koneksi dan pembaruan firmware jarak jauh.

File utama pada folder tersebut adalah Edgent_ESP32.ino, yang memuat identitas template Blynk, nama template, serta versi firmware awal. Pada bagian setup(), komunikasi serial diinisialisasi dan fungsi BlynkEdgent.begin() dipanggil untuk memulai mekanisme Blynk.Edgent. Selanjutnya, fungsi BlynkEdgent.run() dijalankan berulang kali pada bagian loop() agar proses koneksi dan layanan Edgent tetap berlangsung.

File pendukung seperti BlynkEdgent.h, BlynkState.h, ConfigMode.h, ConfigStore.h, Console.h, Indicator.h, OTA.h, ResetButton.h, dan Settings.h mendukung pengelolaan koneksi, konfigurasi perangkat, indikator status, serta mekanisme pembaruan firmware. Kumpulan file ini bekerja sebagai bagian dari lingkungan Blynk.Edgent yang memungkinkan ESP32 menerima firmware baru tanpa harus mengunggah program melalui kabel USB setiap kali ada perubahan.

3.2 File OTA_uafi.ino

File OTA_uafi.ino merupakan program yang berisi fungsi utama praktikum setelah firmware diperbarui. Program ini mengintegrasikan ESP32, driver motor DC, sensor RPM, komunikasi Blynk IoT, dan kontrol melalui komunikasi serial. Pada program tersebut, motor dikendalikan menggunakan sinyal PWM, sedangkan kecepatan putaran motor dipantau melalui pulsa yang diterima dari sensor RPM.

Pada antarmuka Blynk, virtual pin V0 digunakan untuk mengirimkan nilai RPM, V1 untuk menampilkan nilai PWM aktual, V2 untuk menerima pengaturan PWM melalui slider, dan V3 untuk menerima perintah ON/OFF motor. Program juga menyediakan perintah serial berupa 1 untuk menyalakan motor, 0 untuk mematikan motor, + dan - untuk menaikkan atau menurunkan nilai PWM, PWM:angka untuk menetapkan PWM secara langsung, serta R atau r untuk menampilkan status RPM dan PWM pada Serial Monitor.

Program menghitung RPM berdasarkan pulsa sensor yang diterima pada GPIO 13. Perhitungan dilakukan

pada interval sekitar satu detik dengan asumsi nilai HOLES sebesar dua pulsa untuk setiap putaran. Nilai PWM dibatasi pada rentang 0 sampai 255, dengan frekuensi PWM 30 kHz dan resolusi 8-bit. Saat perangkat terhubung kembali ke Blynk, program menetapkan kondisi motor tetap mati terlebih dahulu, kemudian menyinkronkan nilai slider PWM dari dashboard.

3.3 Pembentukan Firmware dan Proses Blynk.air

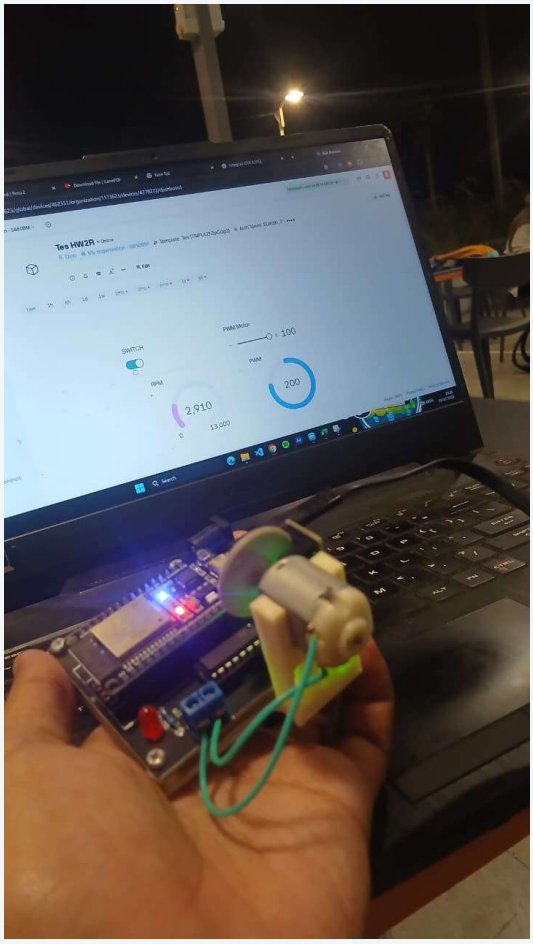
Agar program OTA_uafi.ino dapat dikirim melalui Blynk.Air OTA, kode tersebut dikompilasi terlebih dahulu menggunakan lingkungan pengembangan Arduino yang sesuai dengan board ESP32. Proses kompilasi mengubah kode sumber beserta file pendukung yang diperlukan menjadi berkas firmware berformat biner (.bin). Di dalam arsip praktikum, hasil kompilasi tersedia dengan nama OTA_uafi.ino.bin pada direktori build/esp32.esp32.esp32doit-devkit-v1/.

Berkas biner tersebut merupakan firmware yang dipilih untuk proses pembaruan melalui Blynk.Air, bukan file .ino secara langsung. ESP32 terlebih dahulu harus menjalankan firmware dasar dari folder Edgent_ESP32, terhubung ke jaringan Wi-Fi, dan terhubung ke perangkat Blynk yang sesuai. Setelah firmware biner diunggah melalui mekanisme OTA pada Blynk, perangkat menerima alamat firmware, mengunduh berkas tersebut, memeriksa informasi ukuran dan integritas yang tersedia, lalu menuliskan firmware ke memori perangkat. Jika proses pembaruan berhasil diselesaikan, ESP32 melakukan restart dan menjalankan program baru yang berasal dari OTA_uafi.ino.

Dengan alur tersebut, Edgent_ESP32 berperan sebagai firmware awal yang menyiapkan kemampuan menerima pembaruan, sedangkan OTA_uafi.ino merupakan kode aplikasi yang dikompilasi menjadi berkas biner untuk dipasang sebagai firmware melalui Blynk.Air OTA. Pemisahan ini memungkinkan perubahan pada program pengendalian motor dan pemantauan RPM diterapkan ke ESP32 melalui jaringan, tanpa harus mengunggah ulang program aplikasi secara langsung melalui kabel USB pada setiap pembaruan.

4. Hasil Eksekusi dan Pengujian

FOTO HASIL 1



5. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, pembaruan firmware pada ESP32 dipelajari melalui mekanisme Over-The-Air (OTA) menggunakan platform Blynk.Air. Proses ini diawali dengan memasukkan program dari folder Edgent_ESP32 ke ESP32 sebagai firmware awal agar perangkat dapat terhubung dengan jaringan Wi-Fi dan platform Blynk. Selanjutnya, program OTA_uafi.ino dikompilasi menjadi berkas biner (.bin) yang digunakan sebagai firmware untuk proses pembaruan melalui Blynk.Air OTA. Firmware tersebut berisi program pengendalian motor DC menggunakan PWM, pemantauan kecepatan putaran motor melalui sensor RPM, serta integrasi dengan Blynk IoT. Melalui mekanisme OTA, pembaruan program dapat dilakukan melalui jaringan tanpa harus mengunggah ulang firmware aplikasi menggunakan kabel USB pada setiap perubahan. Dengan demikian, praktikum ini memberikan pemahaman mengenai tahapan persiapan firmware awal, proses kompilasi kode menjadi berkas biner, dan mekanisme pembaruan firmware ESP32 secara nirkabel menggunakan Blynk.Air OTA.